

INTRODUCTION TO ALGORITHMS

Mihir Rao

Contents

1	Introduction	2
1.1	Problems and Algorithms	2
1.2	Correctness	3
1.3	Efficiency and Asymptotic Growth	3
1.4	The Word-RAM Model	4
1.5	Data Structures and Interfaces	5
1.6	A General Design Strategy	5

Lecture 1: Introduction

Algorithm design has three parts: state the problem, find a procedure that solves it, and explain why the procedure is correct and efficient. A problem says which outputs are acceptable. An algorithm says how to find one. A proof checks correctness, and a running-time analysis counts the work.

1.1 Problems and Algorithms

Definition 1.1 [COMPUTATIONAL PROBLEM]. The source describes a computational problem as a “Binary relation from problem inputs to correct outputs” (p. 1). This means that each input is paired with the outputs we accept as correct. A condition, or predicate, lets us check a proposed output.

This relational view allows an input to have several correct outputs. For example, if a collection contains several pairs of people with a shared birthday, returning any one such pair is acceptable. Algorithms are expected to work over a general, arbitrarily large input space rather than for one fixed instance.

Definition 1.2 [ALGORITHM]. The source defines an algorithm as a “Procedure mapping each input to a single output (deterministic)” (p. 1). It solves the problem only if that output is correct for every valid input.

To find a shared birthday, compare each student’s birthday with all earlier ones. Return the first match, or report that none exists. As in the source, assume more possible birthdays than students (including year and time if needed); a fixed 365-day model changes the large- n worst case.

```
1 def find_shared_birthday(students):
2     """Return two names sharing a birthday, or None."""
3     n = len(students)
4     record = StaticArray(n)
5
6     for k in range(n):
7         current_name, current_day = students[k]
8         for i in range(k):
9             previous_name, previous_day = record.get_at(i)
10            if current_day == previous_day:
11                return current_name, previous_name
12            record.set_at(k, students[k])
13
14     return None
```

1.2 Correctness

Testing a few inputs cannot establish correctness on an unbounded input space. A loop or recursive call instead suggests an inductive proof whose statement tracks the portion of the input already processed.

Theorem 1.3. *The birthday-matching algorithm returns a pair if and only if two students in the input share a birthday.*

Proof. First suppose the algorithm returns two names. It does so only after directly comparing their stored birthdays and finding them equal, so every returned pair is valid.

For the converse, induct on the number k of processed students. As the induction hypothesis, assume that if the first k students contain a match, the algorithm has already returned one. At $k = 0$, there are no students and hence no match. Now process student $k + 1$. If a match lies among the first k students, the induction hypothesis applies. Otherwise, any match among the first $k + 1$ students must involve student $k + 1$. The inner loop compares that student's birthday with each of the first k birthdays and therefore finds the match. Thus the claim holds for every prefix, including the complete input. $\square$

The proof separates two obligations that recur throughout algorithm design: **SOUNDNESS**, meaning every produced answer is valid, and **COMPLETENESS**, meaning an answer is found whenever one exists.

1.3 Efficiency and Asymptotic Growth

Wall-clock time depends on hardware, language, compiler, and system load. To compare algorithms independently of those details, analysis counts elementary operations in a stated model of computation and expresses that count as a function of input size, conventionally n .

Definition 1.4 [ASYMPTOTIC BOUNDS]. For eventually nonnegative functions f and g :

- $f(n) = O(g(n))$ if constants $c > 0$ and n_0 exist such that $f(n) \leq c g(n)$ for all $n \geq n_0$;
- $f(n) = \Omega(g(n))$ if constants $c > 0$ and n_0 exist such that $f(n) \geq c g(n)$ for all $n \geq n_0$;
- $f(n) = \Theta(g(n))$ when both bounds hold.

Asymptotic notation leaves out constant factors and lower-order terms to show how quickly the work grows. For large inputs, the progression

$$\Theta(1), \quad \Theta(\log n), \quad \Theta(n), \quad \Theta(n \log n), \quad \Theta(n^2), \quad 2^{\Theta(n)}$$

shows increasingly fast growth. POLYNOMIAL-TIME ALGORITHMS have running time $O(n^c)$ for some constant c and are the course's baseline notion of efficiency, although a lower-degree polynomial is often far more useful in practice.

For birthday matching, allocating the static array costs $\Theta(n)$. On outer iteration k , the inner loop performs at most k constant-time comparisons. The worst-case running time is therefore

$$\begin{aligned} T(n) &= \Theta(n) + \sum_{k=0}^{n-1} \Theta(k+1) \\ &= \Theta(n) + \Theta\left(\frac{n(n+1)}{2}\right) \\ &= \Theta(n^2). \end{aligned}$$

The algorithm is polynomial, but the quadratic scan signals that a better record structure may improve it.

1.4 The Word-RAM Model

An operation count has meaning only after fixing which operations count as constant time. The course uses the *word random-access machine*, or Word-RAM.

Definition 1.5 [WORD-RAM]. A w -bit Word-RAM consists of an addressable memory of w -bit words and a processor that performs arithmetic, comparisons, Boolean and bitwise operations, and reads or writes a constant number of words in $O(1)$ time.

A word must be large enough to encode any relevant memory address. If the algorithm may address n words, this requires

$$w \geq \lceil \log_2 n \rceil.$$

At the same time, treating an arbitrarily large integer as one constant-time word would hide real work. The word-size assumption is what connects the abstract machine to realistic finite hardware. Higher-level languages such as Python provide richer operations, but those operations are ultimately implemented by sequences of Word-RAM steps and may not themselves be constant time.

1.5 Data Structures and Interfaces

Definition 1.6 [DATA STRUCTURE]. A *data structure* stores a nonconstant amount of data and supports a specified collection of operations. That collection of operations is its *INTERFACE*; the representation and algorithms realizing the interface form its implementation.

Separating interface from implementation lets two structures support the same operations with different performance guarantees. A *SEQUENCE* interface organizes items by position: first, last, or *i*th. A *SET* interface organizes them by keys, such as a name or ID, and lets us search using those keys.

The static array is the first concrete example. It stores a fixed number of fixed-width slots in contiguous, addressable memory:

Operation	Meaning	Running time
StaticArray(<i>n</i>)	Allocate and initialize <i>n</i> slots	$\Theta(n)$
get_at(<i>i</i>)	Read the word at index <i>i</i>	$\Theta(1)$
set_at(<i>i</i> , <i>x</i>)	Write word <i>x</i> at index <i>i</i>	$\Theta(1)$

A slot can contain either a value or the address of a larger object. Fixed length makes random access simple but prevents the array from growing; dynamic arrays address that limitation in the next lecture.

1.6 A General Design Strategy

When facing a new algorithms problem, first try to reduce it to a familiar problem whose data structure or algorithm already provides the needed operation. The birthday example asks repeatedly whether a key has appeared, so its quadratic static-array record should eventually be replaced by a data structure supporting faster membership queries.

If no known reduction is available, design a procedure using a standard paradigm: brute force, decrease-and-conquer, divide-and-conquer, dynamic programming, or a greedy incremental construction. In either case the same three questions close the loop:

1. What precisely constitutes a correct output?
2. Why does the algorithm always produce one?
3. How many operations does it require in the chosen model?

Source: MIT OpenCourseWare, MIT 6.006 Introduction to Algorithms, Spring 2020, Lecture 1: Introduction, by Erik Demaine, Jason Ku, and Justin Solomon. [Official lecture resource](#).